perplexity

from datetime import datetime

from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin

db = SQLAlchemy()

Association table: Producer ↔ Category (many-to-many)

producer_category = db.Table(
"producer_category",
db.Column(
"producer_id",
db.Integer,
db.ForeignKey("producer_profile.producer_id"),
primary_key=True,
),
db.Column(
"category_id",
db.Integer,
db.ForeignKey("category.category_id"),
primary_key=True,
),
)

class User(db.Model, UserMixin):
tablename = "user"

```
id = db.Column(db.Integer, primary_key=True)
username = db.Column(db.String(50), unique=True, nullable=False)
email = db.Column(db.String(255), unique=True, nullable=False)
password_hash = db.Column(db.String(255), nullable=False)
first_name = db.Column(db.String(100), nullable=False)
last_name = db.Column(db.String(100), nullable=False)
role = db.Column(db.String(20), nullable=False)  # 'CUSTOMER' | 'PRODUCER'
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


customer_profile = db.relationship(
    "CustomerProfile",
    back_populates="user",
    uselist=False,
```

```
    cascade="all, delete-orphan",
)
producer_profile = db.relationship(
    "ProducerProfile",
    back_populates="user",
    uselist=False,
    cascade="all, delete-orphan",
)
```

class Address(db.Model):
tablename = "address"

```
address_id = db.Column(db.Integer, primary_key=True)
street = db.Column(db.String(255), nullable=False)
city = db.Column(db.String(100), nullable=False)
zip = db.Column(db.String(20), nullable=False)
country = db.Column(db.String(100), nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


customers = db.relationship(
    "CustomerProfile", back_populates="address", lazy="dynamic"
)
producers = db.relationship(
    "ProducerProfile", back_populates="address", lazy="dynamic"
)
```

class CustomerProfile(db.Model):
tablename = "customer_profile"

```
customer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(
    db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True
)
address_id = db.Column(
    db.Integer, db.ForeignKey("address.address_id"), nullable=True
)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


user = db.relationship("User", back_populates="customer_profile")
address = db.relationship("Address", back_populates="customers")
orders = db.relationship("Order", back_populates="customer", lazy="dynamic")
```

class ProducerProfile(db.Model):
tablename = "producer_profile"

```
producer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(
    db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True
)
address_id = db.Column(
```

```
    db.Integer, db.ForeignKey("address.address_id"), nullable=True
)
display_name = db.Column(db.String(255), nullable=False)
legal_name = db.Column(db.String(255), nullable=True)
tax_id = db.Column(db.String(50), nullable=True)
contact_email = db.Column(db.String(255), nullable=False)
contact_phone = db.Column(db.String(50), nullable=True)
verification_status = db.Column(
    db.String(20), nullable=False, default="pending"
)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


user = db.relationship("User", back_populates="producer_profile")
products = db.relationship("Product", back_populates="producer", lazy="dynamic")
address = db.relationship("Address", back_populates="producers")


# NEW: producer can have multiple main categories
categories = db.relationship(
    "Category",
    secondary=producer_category,
    back_populates="producers",
    lazy="select",
)
```

class Category(db.Model):
tablename = "category"

```
category_id = db.Column(db.Integer, primary_key=True)
name = db.Column(db.String(100), nullable=False, unique=True)


products = db.relationship("Product", back_populates="category", lazy="dynamic")
producers = db.relationship(
    "ProducerProfile",
    secondary=producer_category,
    back_populates="categories",
    lazy="dynamic",
)
```

class Product(db.Model):
tablename = "product"

```
product_id = db.Column(db.Integer, primary_key=True)
producer_id = db.Column(
    db.Integer,
    db.ForeignKey("producer_profile.producer_id"),
    nullable=False,
)
category_id = db.Column(
    db.Integer, db.ForeignKey("category.category_id"), nullable=False
)
```

```
name = db.Column(db.String(255), nullable=False)
description = db.Column(db.Text)
price = db.Column(db.Numeric(10, 2), nullable=False)
is_active = db.Column(db.Boolean, default=True, nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


producer = db.relationship("ProducerProfile", back_populates="products")
category = db.relationship("Category", back_populates="products")


cart_items = db.relationship("CartItem", back_populates="product", lazy="dynamic")
order_items = db.relationship(
    "OrderItem", back_populates="product", lazy="dynamic"
)
```

class Cart(db.Model):
tablename = "cart"

```
cart_id = db.Column(db.Integer, primary_key=True)
customer_id = db.Column(
    db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=False
)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
updated_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


customer = db.relationship(
    "CustomerProfile", backref=db.backref("carts", lazy="dynamic")
)
items = db.relationship(
    "CartItem",
    back_populates="cart",
    cascade="all, delete-orphan",
    lazy="dynamic",
)
```

class CartItem(db.Model):
tablename = "cart_item"

```
cart_item_id = db.Column(db.Integer, primary_key=True)
cart_id = db.Column(
    db.Integer, db.ForeignKey("cart.cart_id"), nullable=False
)
product_id = db.Column(
    db.Integer, db.ForeignKey("product.product_id"), nullable=False
)
producer_id = db.Column(
    db.Integer,
    db.ForeignKey("producer_profile.producer_id"),
    nullable=False,
)
quantity = db.Column(db.Integer, nullable=False)
```

```
cart = db.relationship("Cart", back_populates="items")
product = db.relationship("Product", back_populates="cart_items")
producer = db.relationship("ProducerProfile")
```

class Order(db.Model):
tablename = "order"

```
order_id = db.Column(db.Integer, primary_key=True)
customer_id = db.Column(
    db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=False
)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
status = db.Column(db.String(20), nullable=False, default="pending")
total_amount = db.Column(db.Numeric(10, 2), nullable=False)


customer = db.relationship("CustomerProfile", back_populates="orders")
items = db.relationship(
    "OrderItem", back_populates="order", cascade="all, delete-orphan"
)
```

class OrderItem(db.Model):
tablename = "order_item"

```
order_item_id = db.Column(db.Integer, primary_key=True)
order_id = db.Column(
    db.Integer, db.ForeignKey("order.order_id"), nullable=False
)
product_id = db.Column(
    db.Integer, db.ForeignKey("product.product_id"), nullable=False
)
producer_id = db.Column(
    db.Integer,
    db.ForeignKey("producer_profile.producer_id"),
    nullable=False,
)
quantity = db.Column(db.Integer, nullable=False)
unit_price = db.Column(db.Numeric(10, 2), nullable=False)
line_total = db.Column(db.Numeric(10, 2), nullable=False)


order = db.relationship("Order", back_populates="items")
product = db.relationship("Product", back_populates="order_items")
producer = db.relationship("ProducerProfile")
```

this is my data model. i want to create a diagramm a visual of the relational tables

Your Flask-SQLAlchemy models define a comprehensive e-commerce schema for customers, producers, products, carts, and orders. Here's a visual ER diagram of the relational tables and their connections, generated from your code structure.

Core Entities

User serves as the central entity with one-to-one relationships to CustomerProfile or ProducerProfile based on role. Address is shared between CustomerProfile and ProducerProfile (one-to-many).[1] [2]

Product & Category Structure

Products belong to one ProducerProfile and one Category. Producers have many-to-many relationships with Categories via the producer_category association table.[2]

Shopping Flow

Carts link to CustomerProfile with CartItems referencing Product and Producer. Orders follow similar structure with OrderItems capturing snapshot pricing.[3]

Key Relationships Table

Table	Primary Key	Foreign Keys → Targets
user	id	-
address	address_id	-
customer_profile	customer_id	user_id → user.id address_id → address.address_id
producer_profile	producer_id	user_id → user.id address_id → address.address_id
category	category_id	-
product	product_id	producer_id → producer_profile.producer_id category_id → category.category_id
cart	cart_id	customer_id → customer_profile.customer_id
cart_item	cart_item_id	cart_id → cart.cart_id product_id → product.product_id producer_id → producer_profile.producer_id
order	order_id	customer_id → customer_profile.customer_id
order_item	order_item_id	order_id → order.order_id product_id → product.product_id producer_id → producer_profile.producer_id
producer_category	(composite)	producer_id → producer_profile.producer_id category_id → category.category_id [1] [2] [1]

⁂

is there a software that can create tables

Yes, several software tools can create and visualize database tables from your SQLAlchemy models. Many offer visual ER diagram generation perfect for your Flask app schema.

Desktop Tools

DbSchema works offline and generates ER diagrams from SQLAlchemy models or databases - ideal for your multi-table setup with producer_category associations. DBeaver (free) connects to SQLite/PostgreSQL and auto-generates ERDs from existing schemas.[11] [12]

Online Visual Editors

dbdiagram.io uses simple text syntax (like `Table users { id int [pk] }`) to instantly render ERDs - paste your table structure for quick visualization. DrawSQL and DBDesigner.net provide drag-and-drop table creation with SQL export for MySQL/PostgreSQL/SQLite.[13] [14] [15]

Python Libraries

ERAlchemy (`pip install eralchemy`) reads your models.py directly: `eralchemy -i models.py -o schema.png`. It handles relationships, foreign keys, and association tables automatically.[16]

Quick Setup for Your Models

For immediate results, copy your table definitions into dbdiagram.io using their DSL format - it renders your User→CustomerProfile/ProducerProfile polymorphic structure perfectly. Export as PNG/SQL for documentation.[15]

⁂

// Use DBML to define your database structure

// Docs: https://dbml.dbdiagram.io/docs

Table follows {
following_user_id integer
followed_user_id integer
created_at timestamp
}

Table users {
id integer [primary key]
username varchar
role varchar
created_at timestamp
}

Table posts {
id integer [primary key]

title varchar
body text [note: 'Content of the post']
user_id integer [not null]
status varchar
created_at timestamp
}

Ref user_posts: posts.user_id > users.id // many-to-one

Ref: users.id < follows.following_user_id

Ref: users.id < follows.followed_user_id

this is what the software expects as input. reqrite my code to a suitable format: from datetime import datetime
from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin

db = SQLAlchemy()

Association table: Producer ↔ Category (many-to-many)

producer_category = db.Table(
"producer_category",
db.Column(
"producer_id",
db.Integer,
db.ForeignKey("producer_profile.producer_id"),
primary_key=True,
),
db.Column(
"category_id",
db.Integer,
db.ForeignKey("category.category_id"),
primary_key=True,
),
)

class User(db.Model, UserMixin):
tablename = "user"

```
id = db.Column(db.Integer, primary_key=True)
username = db.Column(db.String(50), unique=True, nullable=False)
email = db.Column(db.String(255), unique=True, nullable=False)
password_hash = db.Column(db.String(255), nullable=False)
first_name = db.Column(db.String(100), nullable=False)
last_name = db.Column(db.String(100), nullable=False)
role = db.Column(db.String(20), nullable=False)  # 'CUSTOMER' | 'PRODUCER'
```

```
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


customer_profile = db.relationship(
    "CustomerProfile",
    back_populates="user",
    uselist=False,
    cascade="all, delete-orphan",
)
producer_profile = db.relationship(
    "ProducerProfile",
    back_populates="user",
    uselist=False,
    cascade="all, delete-orphan",
)
```

class Address(db.Model):
tablename = "address"

```
address_id = db.Column(db.Integer, primary_key=True)
street = db.Column(db.String(255), nullable=False)
city = db.Column(db.String(100), nullable=False)
zip = db.Column(db.String(20), nullable=False)
country = db.Column(db.String(100), nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


customers = db.relationship(
    "CustomerProfile", back_populates="address", lazy="dynamic"
)
producers = db.relationship(
    "ProducerProfile", back_populates="address", lazy="dynamic"
)
```

class CustomerProfile(db.Model):
tablename = "customer_profile"

```
customer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(
    db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True
)
address_id = db.Column(
    db.Integer, db.ForeignKey("address.address_id"), nullable=True
)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


user = db.relationship("User", back_populates="customer_profile")
address = db.relationship("Address", back_populates="customers")
orders = db.relationship("Order", back_populates="customer", lazy="dynamic")
```

class ProducerProfile(db.Model):
tablename = "producer_profile"

```
producer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(
    db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True
)
address_id = db.Column(
    db.Integer, db.ForeignKey("address.address_id"), nullable=True
)
display_name = db.Column(db.String(255), nullable=False)
legal_name = db.Column(db.String(255), nullable=True)
tax_id = db.Column(db.String(50), nullable=True)
contact_email = db.Column(db.String(255), nullable=False)
contact_phone = db.Column(db.String(50), nullable=True)
verification_status = db.Column(
    db.String(20), nullable=False, default="pending"
)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


user = db.relationship("User", back_populates="producer_profile")
products = db.relationship("Product", back_populates="producer", lazy="dynamic")
address = db.relationship("Address", back_populates="producers")


# NEW: producer can have multiple main categories
categories = db.relationship(
    "Category",
    secondary=producer_category,
    back_populates="producers",
    lazy="select",
)
```

class Category(db.Model):
tablename = "category"

```
category_id = db.Column(db.Integer, primary_key=True)
name = db.Column(db.String(100), nullable=False, unique=True)


products = db.relationship("Product", back_populates="category", lazy="dynamic")
producers = db.relationship(
    "ProducerProfile",
    secondary=producer_category,
    back_populates="categories",
    lazy="dynamic",
)
```

class Product(db.Model):
tablename = "product"

```
product_id = db.Column(db.Integer, primary_key=True)
producer_id = db.Column(
    db.Integer,
    db.ForeignKey("producer_profile.producer_id"),
    nullable=False,
)
category_id = db.Column(
    db.Integer, db.ForeignKey("category.category_id"), nullable=False
)
name = db.Column(db.String(255), nullable=False)
description = db.Column(db.Text)
price = db.Column(db.Numeric(10, 2), nullable=False)
is_active = db.Column(db.Boolean, default=True, nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


producer = db.relationship("ProducerProfile", back_populates="products")
category = db.relationship("Category", back_populates="products")


cart_items = db.relationship("CartItem", back_populates="product", lazy="dynamic")
order_items = db.relationship(
    "OrderItem", back_populates="product", lazy="dynamic"
)
```

class Cart(db.Model):
tablename = "cart"

```
cart_id = db.Column(db.Integer, primary_key=True)
customer_id = db.Column(
    db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=False
)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
updated_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)


customer = db.relationship(
    "CustomerProfile", backref=db.backref("carts", lazy="dynamic")
)
items = db.relationship(
    "CartItem",
    back_populates="cart",
    cascade="all, delete-orphan",
    lazy="dynamic",
)
```

class CartItem(db.Model):
tablename = "cart_item"

```
cart_item_id = db.Column(db.Integer, primary_key=True)
cart_id = db.Column(
    db.Integer, db.ForeignKey("cart.cart_id"), nullable=False
```

```
)
product_id = db.Column(
    db.Integer, db.ForeignKey("product.product_id"), nullable=False
)
producer_id = db.Column(
    db.Integer,
    db.ForeignKey("producer_profile.producer_id"),
    nullable=False,
)
quantity = db.Column(db.Integer, nullable=False)


cart = db.relationship("Cart", back_populates="items")
product = db.relationship("Product", back_populates="cart_items")
producer = db.relationship("ProducerProfile")
```

class Order(db.Model):
tablename = "order"

```
order_id = db.Column(db.Integer, primary_key=True)
customer_id = db.Column(
    db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=False
)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
status = db.Column(db.String(20), nullable=False, default="pending")
total_amount = db.Column(db.Numeric(10, 2), nullable=False)


customer = db.relationship("CustomerProfile", back_populates="orders")
items = db.relationship(
    "OrderItem", back_populates="order", cascade="all, delete-orphan"
)
```

class OrderItem(db.Model):
tablename = "order_item"

```
order_item_id = db.Column(db.Integer, primary_key=True)
order_id = db.Column(
    db.Integer, db.ForeignKey("order.order_id"), nullable=False
)
product_id = db.Column(
    db.Integer, db.ForeignKey("product.product_id"), nullable=False
)
producer_id = db.Column(
    db.Integer,
    db.ForeignKey("producer_profile.producer_id"),
    nullable=False,
)
quantity = db.Column(db.Integer, nullable=False)
unit_price = db.Column(db.Numeric(10, 2), nullable=False)
line_total = db.Column(db.Numeric(10, 2), nullable=False)
```

```
order = db.relationship("Order", back_populates="items")
product = db.relationship("Product", back_populates="order_items")
producer = db.relationship("ProducerProfile")
```

```
// FoodLoop E-Commerce Database Schema
// Generated from Flask-SQLAlchemy models for dbdiagram.io

Table user {
  id integer [primary key]
  username varchar(50) [unique, not null]
  email varchar(255) [unique, not null]
  password_hash varchar(255) [not null]
  first_name varchar(100) [not null]
  last_name varchar(100) [not null]
  role varchar(20) [not null] // 'CUSTOMER' | 'PRODUCER'
  created_at timestamp [not null, default: `now()`]
}

Table address {
  address_id integer [primary key]
  street varchar(255) [not null]
  city varchar(100) [not null]
  zip varchar(20) [not null]
  country varchar(100) [not null]
  created_at timestamp [not null, default: `now()`]
}

Table customer_profile {
  customer_id integer [primary key]
  user_id integer [not null, unique]
  address_id integer
  created_at timestamp [not null, default: `now()`]
}

Table producer_profile {
  producer_id integer [primary key]
  user_id integer [not null, unique]
  address_id integer
  display_name varchar(255) [not null]
  legal_name varchar(255)
  tax_id varchar(50)
  contact_email varchar(255) [not null]
  contact_phone varchar(50)
  verification_status varchar(20) [not null, default: `'pending'`]
  created_at timestamp [not null, default: `now()`]
}

Table category {
  category_id integer [primary key]
  name varchar(100) [not null, unique]
}

Table product {
  product_id integer [primary key]
  producer_id integer [not null]
```

```
  category_id integer [not null]
  name varchar(255) [not null]
  description text
  price decimal(10,2) [not null]
  is_active boolean [not null, default: `true`]
  created_at timestamp [not null, default: `now()`]
}

Table cart {
  cart_id integer [primary key]
  customer_id integer [not null]
  created_at timestamp [not null, default: `now()`]
  updated_at timestamp [not null, default: `now()`]
}

Table cart_item {
  cart_item_id integer [primary key]
  cart_id integer [not null]
  product_id integer [not null]
  producer_id integer [not null]
  quantity integer [not null]
}

Table order {
  order_id integer [primary key]
  customer_id integer [not null]
  created_at timestamp [not null, default: `now()`]
  status varchar(20) [not null, default: `'pending'`]
  total_amount decimal(10,2) [not null]
}

Table order_item {
  order_item_id integer [primary key]
  order_id integer [not null]
  product_id integer [not null]
  producer_id integer [not null]
  quantity integer [not null]
  unit_price decimal(10,2) [not null]
  line_total decimal(10,2) [not null]
}

// Association table for Producer <-> Category (many-to-many)
Table producer_category {
  producer_id integer [pk]
  category_id integer [pk]
}

// Core relationships
Ref: customer_profile.user_id > user.id // 1:1
Ref: producer_profile.user_id > user.id // 1:1
Ref: customer_profile.address_id > address.address_id // 1:N optional
Ref: producer_profile.address_id > address.address_id // 1:N optional

// Product relationships
Ref: product.producer_id > producer_profile.producer_id // 1:N
Ref: product.category_id > category.category_id // 1:N
```

```
// Shopping cart relationships
Ref: cart.customer_id > customer_profile.customer_id // 1:N
Ref: cart_item.cart_id > cart.cart_id // 1:N
Ref: cart_item.product_id > product.product_id // N:1
Ref: cart_item.producer_id > producer_profile.producer_id // N:1

// Order relationships
Ref: order.customer_id > customer_profile.customer_id // 1:N
Ref: order_item.order_id > order.order_id // 1:N
Ref: order_item.product_id > product.product_id // N:1
Ref: order_item.producer_id > producer_profile.producer_id // N:1

// Many-to-many relationships
Ref: producer_category.producer_id > producer_profile.producer_id
Ref: producer_category.category_id > category.category_id
```

Copy this entire block and paste it directly into dbdiagram.io - it will instantly render your complete ER diagram with all relationships, foreign keys, and the producer_category association table properly connected. The polymorphic User→CustomerProfile/ProducerProfile structure and shopping cart/order flows are all preserved.

should there be a relation between cart and order? technical cart is just a space where i safe products temporarily and cart content becomes the order once it is placed

No, there shouldn't be a direct foreign key relationship between Cart and Order tables in your schema. Your current design is correct for a typical e-commerce flow.

Why No Direct Cart→Order Link

Carts serve as temporary workspaces. When an order is placed:

- Cart content gets **copied** to Order/OrderItem tables (with snapshot pricing)
- Cart can then be cleared/updated for the next shopping session
- Orders remain immutable for accounting/audit purposes

A direct FK would incorrectly imply one cart = one order, but customers often:

- Place multiple orders from different carts/sessions
- Abandon carts entirely
- Merge multiple carts before checkout

Current Flow (Correct)

```
Customer adds → Cart → Checkout → Order created → Cart cleared
                                ↓
                          OrderItem snapshots
```

Your Schema Handles This Perfectly

```
Cart → CartItem → Product/Producer (temporary)
↓
Order → OrderItem → Product/Producer (permanent, with unit_price snapshot)
```

Optional Enhancement

Consider adding `order_id` to Cart **only if** you want to track "which cart created this order" for analytics:

```
class Cart(db.Model):
    # ... existing fields
    order_id = db.Column(db.Integer, db.ForeignKey("order.order_id"), nullable=True)
```

But even then, make it nullable/optional since most carts never become orders. Your current design follows e-commerce best practices.

1. https://stackoverflow.com/questions/44981986/sqlalchemy-er-diagram-in-python-3
2. https://github.com/eralchemy/eralchemy
3. https://pypi.org/project/sqlalchemy-data-model-visualizer/
4. https://github.com/sqlalchemy/sqlalchemy/wiki/SchemaDisplay
5. https://www.reddit.com/r/Python/comments/17i2u7y/easily_visualize_your_sqlalchemy_data_models_in_a/
6. http://slawomir.net/2019/11/09/savis/
7. https://blog.tedivm.com/open-source/2024/01/visualizing-sqlalchemy-databases-using-paracelsus/
8. https://www.youtube.com/watch?v=I_431Ef_NwA
9. https://sourceforge.net/projects/eralchemy.mirror/
10. https://pypi.org/project/eralchemy/0.0.11/
11. https://dbschema.com/blog/design/best-database-design-tools-2025/
12. https://dbschema.com
13. https://drawsql.app
14. https://www.dbdesigner.net
15. https://dbdiagram.io
16. https://github.com/eralchemy/eralchemy
17. https://www.canva.com/graphs/tables/

18. https://www.atlassian.com/data/databases/7-free-database-diagramming-tools-for-busy-data-folks
19. https://www.reddit.com/r/Database/comments/yxgvkm/need_recommendations_for_userfriendly_database/
20. https://www.beekeeperstudio.io
21. https://www.adobe.com/express/create/table